Task 4 - Lifecycle: verified boot, SPDM identity and secure update

PSoC Control C3M6 (KIT_PSC3M6_EVAL) - the SPDM 1.4 ML-DSA-87 / ML-KEM-1024 responder now runs **behind the Edge Protect Bootloader**, which verifies it with ML-DSA-87 on every boot and before every update; the SPDM measurements report the image EPBL verified.

1. Summary

Area	Result	Evidence
Firmware update path (responder image)	16/16 steps as expected	valid update v1.0.0 -> v1.1.0 installed; rollback, corrupted payload, modified header, rogue ML-DSA-87 key rejected; tampered primary does not boot; recovery by signed update v2.0.0
Reconnect after update	PASS (3 measure + 3 exchange after updates)	same PAS identity before/after every update: no re-provisioning needed
Boot identity -> SPDM	measurements = verified image	block 1/2/3 recomputed from the release files MATCH after every boot (4 of 4)
Credential update path	rotation PASS (2 exchanges); 9/10 device checks as expected	new on-device ML-DSA-87 key + cert; unsupported alg, oversize chain, foreign leaf (proof-of-possession), SHA-256 RootHash rejected; 1 finding (V5b, by design before freeze)
Cost	EPBL verify 80 ms, reset->responder 157 ms	update install 4.95 s (erase 3.55 s, copy 1.15 s); EPBL 47,244 B flash unchanged

2. Architecture: one chain from reset to the SPDM session

The responder build gained a BSP variant (`TARGET=APP_KIT_PSC3M6_EVAL_EPBL`) with the EPBL flash map; secure image after a 0x400 B MCUboot header, non-secure image inside the same signed image. Memory regions and the MPC configuration were regenerated with the Device Configurator CLI. The PAS (SPDM identity) keeps its address, so the identity provisioned in Task 3 survived the migration and every update.

Area	Address range	Content / who writes it
EPBL	0x32000000-0x3201B000 (108 KB)	Edge Protect Bootloader, ML-DSA-87 validation key compiled in
Primary slot	0x3201B000-0x32048000 (180 KB)	header + responder S (0x3201B400) + NS (0x32034000) + TLVs (SHA-512, key hash, 4,627 B ML-DSA-87 signature); written only by EPBL (install) or debugger
PAS	0x32048000-0x3204E000 (24 KB)	SPDM identity: ML-DSA-87 key slot 0 (generated on device), cert chain slot 1 (15,187 B); written only by the provisioning service
Upgrade slot	0x3204E000-0x3207B000 (180 KB)	candidate image; verified by EPBL before install, erased after install or rejection

Stage	Verifies	Algorithm	Status on this board
ROM_BOOT (immutable)	SFLASH / FLASH_BOOT	Infineon keys	classical, Infineon root
FLASH_BOOT	EPBL (with SECURE_APP policy + OEM key)	ECDSA P-256/384/521	SIMPLE_APP policy: EPBL launched WITHOUT authentication (no OEM key on the shared board)
EPBL	responder image, every boot + before install	ML-DSA-87, SHA-512 (HashML-DSA)	enforced: 80 ms per verification
Responder SPE	-- (signs with the PAS identity key)	ML-DSA-87 sign, ML-KEM-1024 encaps	reports EPBL-verified image digest/version as signed SPDM measurements
Host requester	cert chain to pinned ML-DSA-87 root; signatures; measurements vs reference values	ML-DSA-87, ML-KEM-1024, SHA-384, AES-256-GCM	tools/task4_verify_meas.py recomputes the reference values

3. Firmware update path (responder image through EPBL)

Runner `tools/task4_epbl_spdm_tests.ps1` -> results/task4_EPBL_SPDM_20261111_2059. Images are loaded with the debugger into the slot (the in-field equivalent is the application writing the upgrade slot); EPBL decides at reset. All images contain the same code with different MCUboot versions (version, digest and rollback behaviour are what is tested).

Step	Action	Expected	Result	EPBL / requester evidence
T0_boot_v1_0_0	EPBL + signed v1.0.0 in primary, empty upgrade slot	BOOTED	BOOTED	20:59:22.461 [I] === PSOC Control C3 SPDM Responder (non-secure) ===

Step	Action	Expected	Result	EPBL / requester evidence
T0_measure_v1_0_0	signed GET_MEASUREMENTS	PASS	PASS	block 3: MutableFirmwareSecurityVersion (raw) (8 B) 0000000100000000 | block4='PSC3M6-SPDM-P4 fw 1.0.0+0 EPBL'
T0_exchange_v1_0_0	ML-KEM session + encrypted message	PASS	PASS	exchange: OK (encrypted app message, sent 20 B, received 20 B)
T1_valid_update_v1_1_0	signed v1.1.0 written to upgrade slot, reset	UPDATED	UPDATED	21:18:01.336 [I] === PSOC Control C3 SPDM Responder (non-secure) ===
T1_measure_v1_1_0	measurements after update	PASS	PASS	block 3: MutableFirmwareSecurityVersion (raw) (8 B) 0000010100000000 | block4='PSC3M6-SPDM-P4 fw 1.1.0+0 EPBL'
T1_exchange_after_update	reconnect after update	PASS	PASS	exchange: OK (encrypted app message, sent 19 B, received 19 B)
T2_rollback_v1_0_0	signed (genuine key) v1.0.0 as update	REJECTED	REJECTED	[ERR] insufficient version in secondary slot (downgrade prevention)
T3_corrupt_payload	v1.2.0 update, 1 payload byte flipped	REJECTED	REJECTED	validate returned 0xffffffff: SHA-512 image hash mismatch -> secondary not valid
T4_modified_header	v1.2.0 update, header revision byte changed	REJECTED	REJECTED	validate returned 0xffffffff: SHA-512 image hash mismatch -> secondary not valid
T5_rogue_key	v1.2.0 signed with a rogue ML-DSA-87 key	REJECTED	REJECTED	validate returned 0x2e: no key matches the key-hash TLV -> secondary not valid
T6_measure_still_v1_1_0	measurements after the 4 rejected updates	PASS	PASS	block 3: MutableFirmwareSecurityVersion (raw) (8 B) 0000010100000000 | block4='PSC3M6-SPDM-P4 fw 1.1.0+0 EPBL'
T7_tampered_primary	1 byte flipped in the installed primary image	NOBOOT	NOBOOT	validate returned 0xffffffff: SHA-512 image hash mismatch -> secondary not valid
T7_connect_no_app	SPDM connect while EPBL halted	REJECT	REJECT	spdm-requester: error: unrecognized arguments: --root C:\Users\kpato\KP\Hackathon\results\task3_neg_roots\good_root.der
T8_recovery_update_v2_0_0	signed v2.0.0 to upgrade slot (primary invalid)	UPDATED	UPDATED	21:40:36.946 [I] === PSOC Control C3 SPDM Responder (non-secure) ===
T8_measure_v2_0_0	measurements after recovery	PASS	PASS	block 3: MutableFirmwareSecurityVersion (raw) (8 B) 0000000200000000 | block4='PSC3M6-SPDM-P4 fw 2.0.0+0 EPBL'
T8_exchange_after_recovery	reconnect after recovery	PASS	PASS	exchange: OK (encrypted app message, sent 16 B, received 16 B)

4. Boot identity and SPDM identity

Answer to the brief: the runtime SPDM identity is anchored in BOTH. The identity *key* is a separately provisioned credential (ML-DSA-87 key generated inside the SPE, PAS slot 0, never exported; certified by the ML-DSA-87 OEM CA). The *firmware state* is anchored in the verified boot chain: the responder reports, in ML-DSA-87-signed GET_MEASUREMENTS, the digest of the EPBL boot stage (block 1), the digest of exactly the bytes EPBL hashed and verified (block 2) and the MCUboot version (block 3). A verifier recomputes these from the release files (`tools/task4_verify_meas.py`):

Measurement	Image	block 1 EPBL	block 2 image	block 3 SVN	block 4 (device string)
T0_measure_v1_0_0	resp_v1_0_0_boot.hex	MATCH	MATCH	MATCH	PSC3M6-SPDM-P4 fw 1.0.0+0 EPBL
T1_measure_v1_1_0	resp_v1_1_0_update.hex	MATCH	MATCH	MATCH	PSC3M6-SPDM-P4 fw 1.1.0+0 EPBL
T6_measure_still_v1_1_0	resp_v1_1_0_update.hex	MATCH	MATCH	MATCH	PSC3M6-SPDM-P4 fw 1.1.0+0 EPBL
T8_measure_v2_0_0	resp_v2_0_0_update.hex	MATCH	MATCH	MATCH	PSC3M6-SPDM-P4 fw 2.0.0+0 EPBL

Consequence	Why / mitigation
Firmware update does not change the identity	PAS is outside both slots: reconnect after update works without re-provisioning (T1, T8). The verifier sees the new version only through the measurements.
Identity does not prove WHICH firmware runs	any image EPBL accepts can use the key -> the verifier must check the signed measurements against reference values (and a minimum SVN), not only the certificate.
An unsigned/tampered image cannot use the key	it never runs (T3-T5, T7). Remaining paths: debugger access (DAP open in DEVELOPMENT LCS) and the unauthenticated BootROM->EPBL step.
Remote rollback detection	local downgrade prevention guards only the update path (Task 2 L1: debugger can write an old signed image to primary); block 3 exposes the running version, so a verifier can refuse old versions.
Stronger binding (next step)	DICE-style: derive the attestation key from a CDI over the EPBL-measured image, so a different image gets a different key; or bind measurements into the certificate (TCB info extension).

5. Credential update path (SPDM identity in the PAS)

Provisioning service (ML-DSA-87 suite) over the same I2C/DFU link: erase -> on-device gen-key -> CSR -> OEM CA signs -> put-cert (SPDM chain, SHA-384 RootHash) -> read-back -> prove-cert (proof of possession). Device-side checks: results/task4_EDGE_PROV_20261111_1903; rotation + reconnect: results/task4_EDGE_RESTORE_20261111_1912 (a third repeat exchange was cut by a PC power-off).

Test	Expected	Result	Evidence
V1_info	PASS	OK	prov_cmd_status : PROV_OK
V2a_erase_cert	PASS	OK	prov_cmd_status : PROV_OK
V2b_erase_key	PASS	OK	prov_cmd_status : PROV_OK
V3_genkey_mldsa65_unsupported	REJECT	OK	prov_cmd_status : PROV_ERR_ALG_UNSUPPORTED
V4_genkey_ecdsa384_not_built	REJECT	OK	prov_cmd_status : PROV_ERR_ALG_UNSUPPORTED
V5_genkey_mldsa87	PASS	OK	prov_cmd_status : PROV_OK
V5b_genkey_occupied_slot	REJECT	UNEXPECTED	prov_cmd_status : PROV_OK
V6_oversize_chain	REJECT	OK	prov_cmd_status : PROV_ERR_LENGTH
V7b_prove_foreign_leaf	REJECT	OK	FAIL: signature verification failed.
V9_roothash_sha256_vs_sha384	REJECT	OK	spdm-requester.exe : error: get_certificate failed: SPDM_REQ_EAUTH
R_restore_exchange	PASS	OK	exchange: OK (encrypted app message, sent 11 B, received 11 B)
R_repeat_exchange_2	PASS	OK	exchange: OK (encrypted app message, sent 8 B, received 8 B)

V5b finding: gen-key on an occupied slot is accepted and replaces the key. This is the provisioning app's design while the PAS is not frozen (a frozen device answers PROV_ERR_FROZEN, provisioning-app self-test). Consequence: credential immutability in the field depends on `--freeze` (not used on the shared board), a closed debug port, and the provisioning image never being bootable in the field - with EPBL enforcing ML-DSA-87, an unsigned provisioning image cannot run; a signed one is an OEM decision. **V7:** the device stores a leaf whose key is not the slot key (no on-device X.509 parsing, design D4), but prove-cert fails, and so would SPDM CHALLENGE - provisioning must gate on prove-cert.

6. Migration strategy for devices that start with ECC credentials

Hybrid ECC+PQC is excluded by the brief, so each plane is cut over per device; the requester fleet accepts both roots only during the transition window. Steps 1-4 were executed on this board (Task 1 -> Tasks 2/3/4).

#	Step	Authenticated by	Shown
1	Field firmware update to an image that contains the PQC code (P4 responder, ML-DSA/ML-KEM in the SPE) and still identifies with ECC	existing ECDSA-P384 EPBL path	Task 1/2 (ECDSA update path)
2	EPBL validation key ECDSA-P384 -> ML-DSA-87 (EPBL image change)	BootROM/FLASH_BOOT (OEM key, ECDSA) - the only way EPBL itself can be replaced securely; here by debugger	Task 2 (EPBL reflash)
3	New identity: on-device ML-DSA-87 keygen + CSR, CA-signed by the new ML-DSA-87 OEM CA; production: the CSR request is authenticated with the still-valid ECC identity (SPDM session with the old key) so the CA knows it is the same device	ECC identity (bridge), then ML-DSA-87 CA	Task 3 P2->P4 re-provisioning, Task 4 rotation
4	Switch the responder to the PQC-only profile (no classical algorithm offered -> no downgrade)	ML-DSA-87 EPBL	Task 3/4 (P4 image behind EPBL)
5	Requesters drop the ECC root (A3: P4 requester rejects a classical root); revoke ECC CA	operator policy	Task 3 A0/A3
6	Freeze PAS, close DAP, LCS SECURE; HW anti-rollback counter	irreversible - not on shared board	documented

7. What remains classical and what must change for full quantum resistance

Item	Status	What must change
BootROM / FLASH_BOOT -> EPBL	classical root (ECDSA OEM key) and NOT enabled on this board (SIMPLE_APP, no OEM key)	provision OEM key + SECURE_APP policy (needs EPSS-signed OEM certificate); long term a ROM with ML-DSA/LMS. Until then EPBL - and the ML-DSA-87 key inside it - is only as trustworthy as an ECDSA (or no) check

Item	Status	What must change
EPBL	PQC: ML-DSA-87 / SHA-512, every boot and every update	EPBL is not field-updatable in this map
SPDM authentication / key exchange	PQC only: ML-DSA-87, ML-KEM-1024, SHA-384, AES-256-GCM	-
SPDM identity provisioning channel	unauthenticated factory channel; PAS not frozen	freeze; authenticated re-provisioning
Rollback	software downgrade prevention (update path only); version visible in SPDM block 3	hw_rollback_prot (eFuse counter) + closed DAP
Debug port	open (DEVELOPMENT LCS)	close DAP / LCS SECURE
Update delivery	debugger writes the upgrade slot	in-band update over the SPDM/DFU channel
Image content	test images differ only in version	real releases differ in code; same mechanism

8. Resource and performance impact of the integration

Metric	Task 3 (standalone, no EPBL)	Task 4 (behind EPBL)	Note
Boot verification	none	80 ms ML-DSA-87 + SHA-512 over 165 KB	CE243041 app in Task 2: 61 ms over 128 KB
Reset -> responder start	immediate	157 ms	
Update install	n/a (debugger)	4.95 s	erase 180 KB 3.55 s, copy 1.15 s, 2 verifications
Flash	288 KB reserved for the responder	108 KB EPBL + 2 x 180 KB slots	responder uses 145 KB; 20 KB free
Image metadata	-	0x400 B header + 4,771 B TLVs per slot	
SPDM session (transfers / bytes)	214 / 28,199 B	215 / 28,330 B	unchanged protocol cost
Measurement digests	placeholders	SHA-384 over 108 KB + 165 KB, once per boot (cached)	within the ~0.3-0.6 s device time of a measure op

9. Reproduction

```
EPBL (spdm map) : make -C epbl/bl build MEMORY_MAP=<abs path>/epbl/bl/platforms/PSC3_M6/memory_maps/spdm_overwrite_single_
Responder       : make -C psoc-spdm-responder build TARGET=APP_KIT_PSC3M6_EVAL_EPBL SPDM_PROFILE=P4
Sign / variants : tools\task4_epbl_spdm_tests.ps1 -Phase PREP   (tools/resp_epbl_sign.py, hex_pad.py, hex_flip.py)
Board run       : tools\task4_epbl_spdm_tests.ps1 -Phase BOARD  (T0..T8, ~55 min)
Reference values: py -3.13 -I tools/task4_verify_meas.py results/task4_boot/epbl_full.hex <image.hex> <measure.txt>
Credentials     : tools\task3_edge_tests.ps1 -Phase PROV -Prefix task4 ; -Phase RESTORE -Prefix task4
Offline tests   : py -3.13 -m unittest discover -s tools/tests
Patch / BSP     : patches/psoc-spdm-responder_task4.patch + patches/bsp_overlay_TARGET_APP_KIT_PSC3M6_EVAL_EPBL/
Runs            : results/task4_EPBL_SPDM_20261111_2059, results/task4_EDGE_PROV_20261111_1903, results/task4_EDGE_RESTORE
```